

Machine Learning for Natural Language Processing

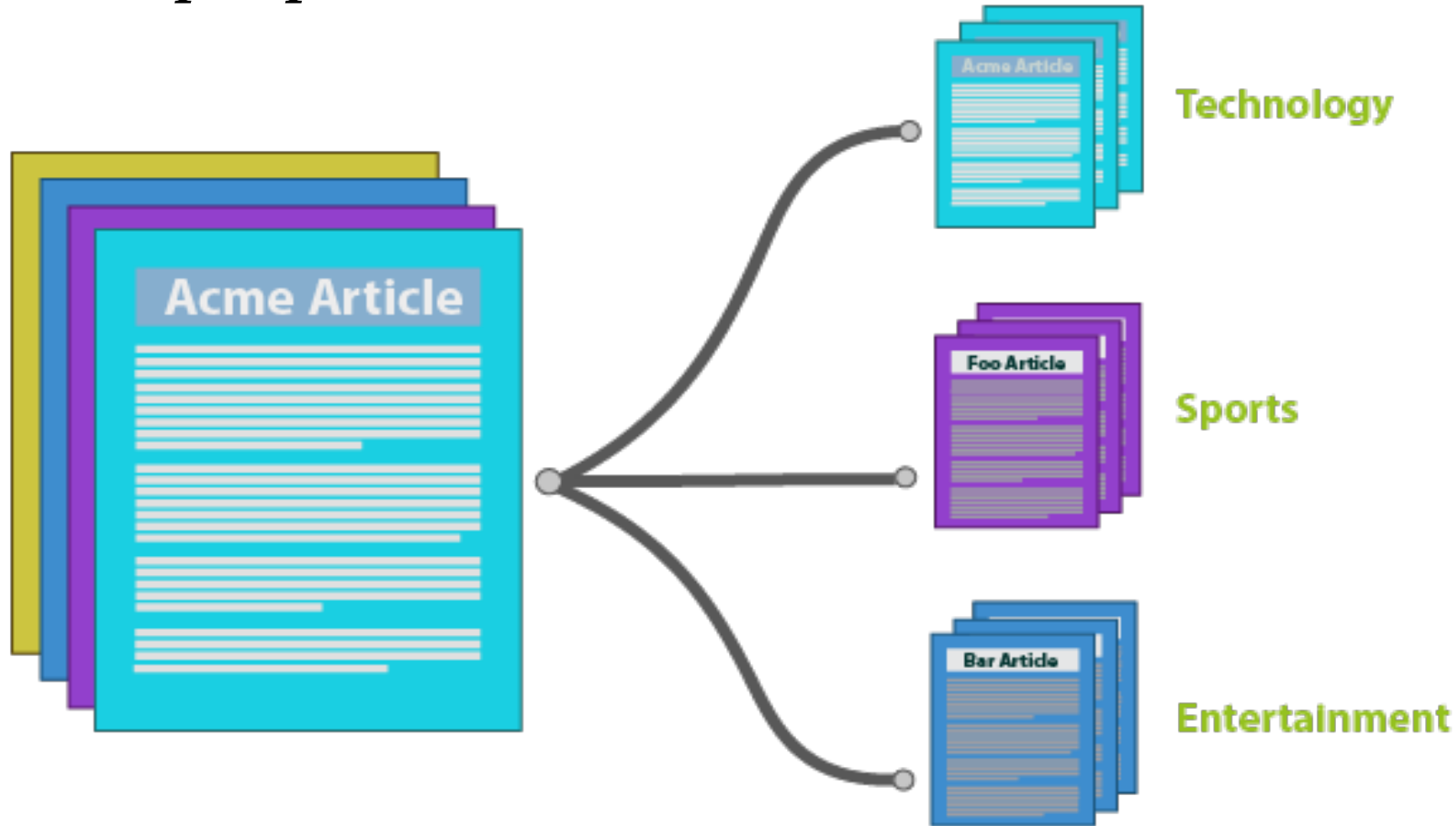
Luca Putelli

Natural Language Processing

- Definition: *the application of computational techniques to the analysis and synthesis of natural language and speech.*
- But natural language is designed to make human communication efficient, so:
 - We use a lot of common sense knowledge
 - We keep a lot of ambiguities that can be resolved by context
- NLP tries to deal with these differences in order to understand language

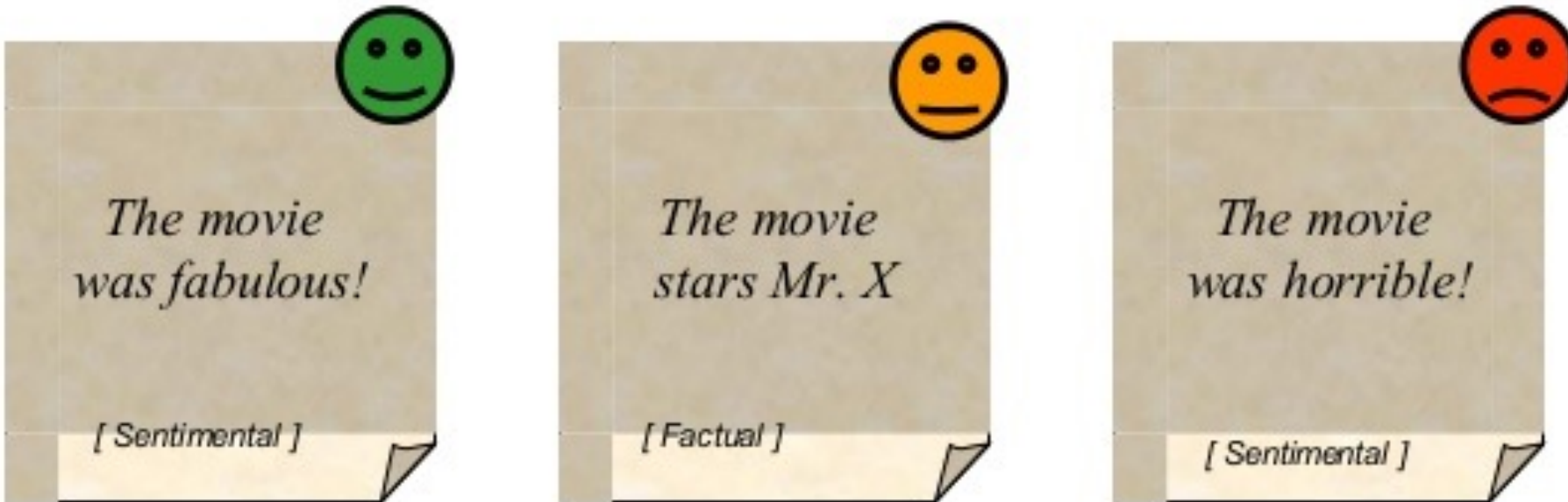
Main tasks: Classification

- Definition: *labeling automatically text documents on the basis of topic and purpose*

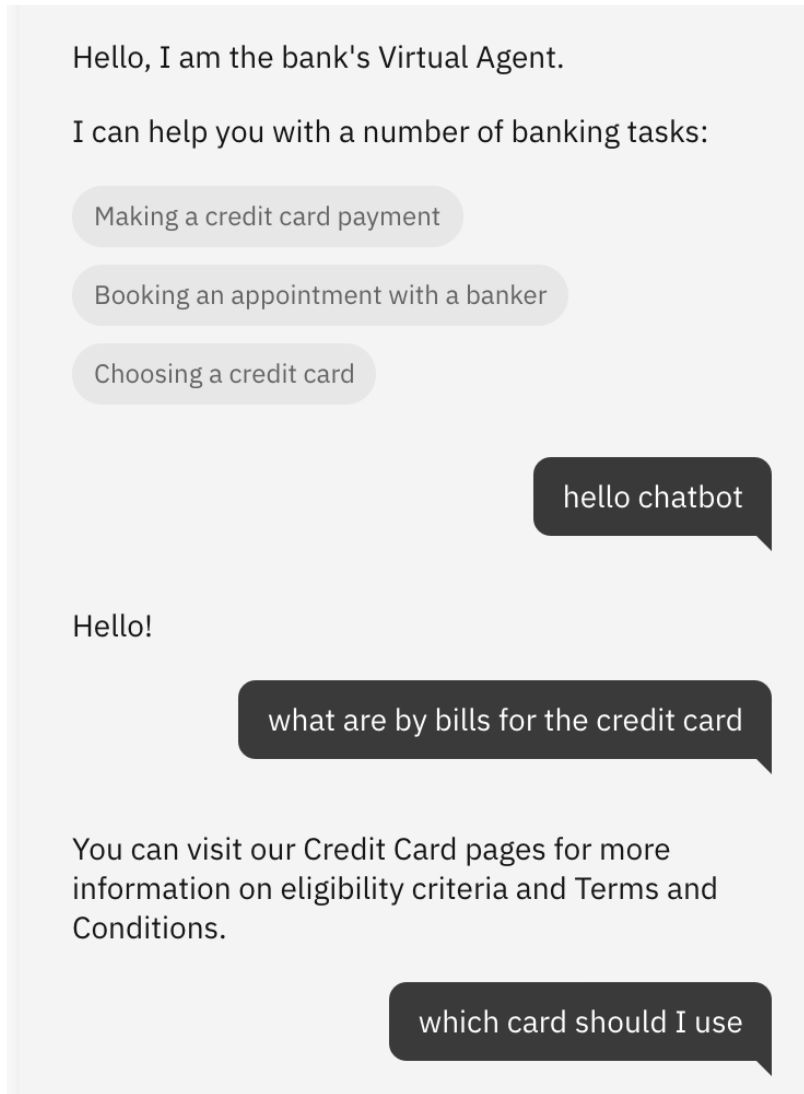


Main tasks: Sentiment Analysis

- Definition: *measures the inclination of people's opinions through natural language processing (NLP) and text analysis*



Main tasks: Chatbots

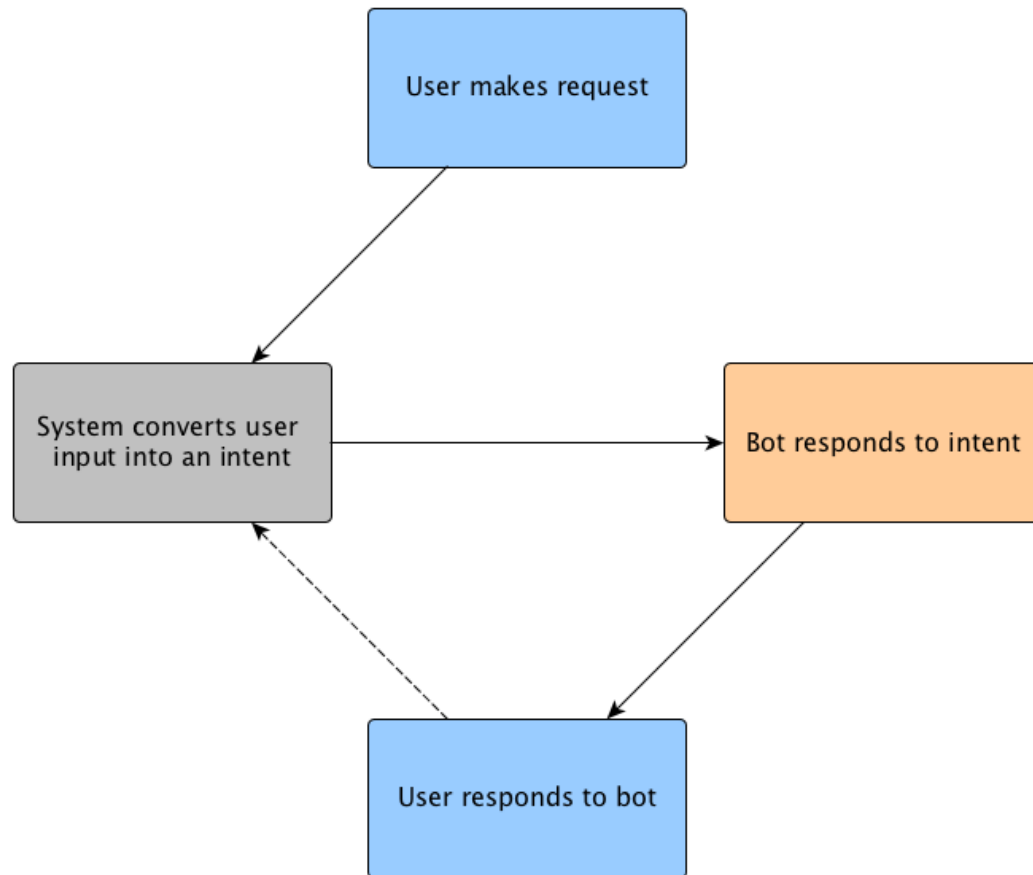


Definition: *software for conducting an on-line conversation with a human agent*

It can be used for many application tasks

ChatGPT is a very huge one of them.

Main tasks: Chatbots

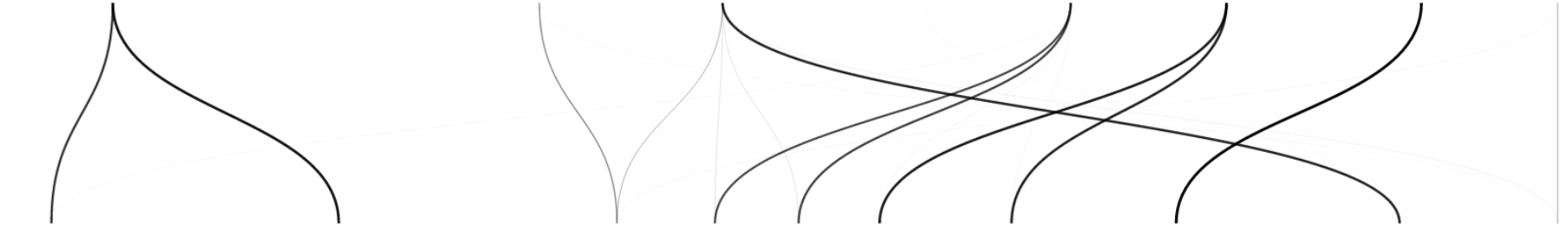


- **Intent:** a general indication of what the user is saying («Greetings», «Assistance», «Issues», etc.)
- After recognising the intent, **the chatbot answers to the user with the information he or she requested**

Main tasks: Machine translation

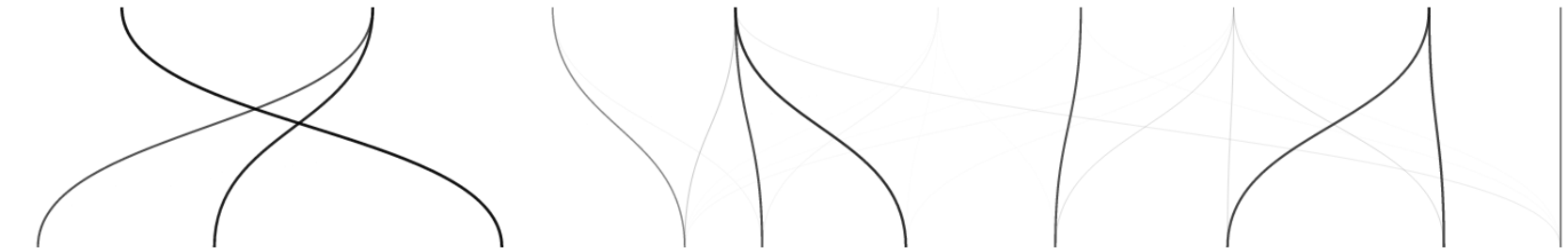
- Definition: *translate text from one language to another*

Economic growth has slowed down in recent years .



Das Wirtschaftswachstum hat sich in den letzten Jahren verlangsamt .

Economic growth has slowed down in recent years .



La croissance économique s' est ralentie ces dernières années .

How to deal with text?

- A word is not only a string: **word representation**
 - Deal with gender, number, word conjugation, semantics, synonyms, contraries in order to represent a word with its meaning
- We would also like to use standard algorithms (Decision Trees, Naive Bayes, Neural Networks) in order to classify documents, so we have to find a way to have **document representation**
- **Feature selection**: select the relevant sections in a document («The movie stars Mr. X» describes an opinion?)

The most simple way: One-hot vector

"a"	"abbreviations"		"zoology"	"zoom"
1	0		0	0
0	1		0	0
0	0		0	0
.	.	.	.	.
.	.		.	.
.	.		.	.
0	0		0	0
0	0		1	0
0	0		0	1

- Every word has an ID
- The vector is filled with 0s except for a 1 at the position associated with the ID
- All words are equally different from each other: «a» and «an» have the same difference as «a» and «zoology»
- This representation doesn't deal with number («shoe» and «shoes» are two completely different words), genre, synonyms

Bag of Words

T_j : j-th term

D_i : i-th document

w_{ij} : 1 if T_j appears in D_i
0 otherwise

	T_1	T_2	...	T_t
D_1	w_{11}	w_{21}	...	w_{t1}
D_2	w_{12}	w_{22}	...	w_{t2}
$\vdots$	$\vdots$	$\vdots$		$\vdots$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
D_n	w_{1n}	w_{2n}	...	w_{tn}

the dog is on the table

0	0	1	1	0	1	1	1
are	cat	dog	is	now	on	table	the

- It's the most common way to represent documents for classification
- We produce a document-term matrix
- Every row represents the words contained in a document
- Every column represent in which documents the word appears

Classification with Bag of Words

	linux	modem	the	clutch	steering	petrol
D1	1	1	1	0	0	0
D2	1	1	1	0	0	0
D3	0	0	1	1	1	1
D4	1	0	1	1	1	1

Red: documents about information technology

Blue: documents about cars

- Idea: similar documents contain similar words
- Document-term matrix can be the input of a machine learning algorithm (Decision Tree, SVM, Neural Network)

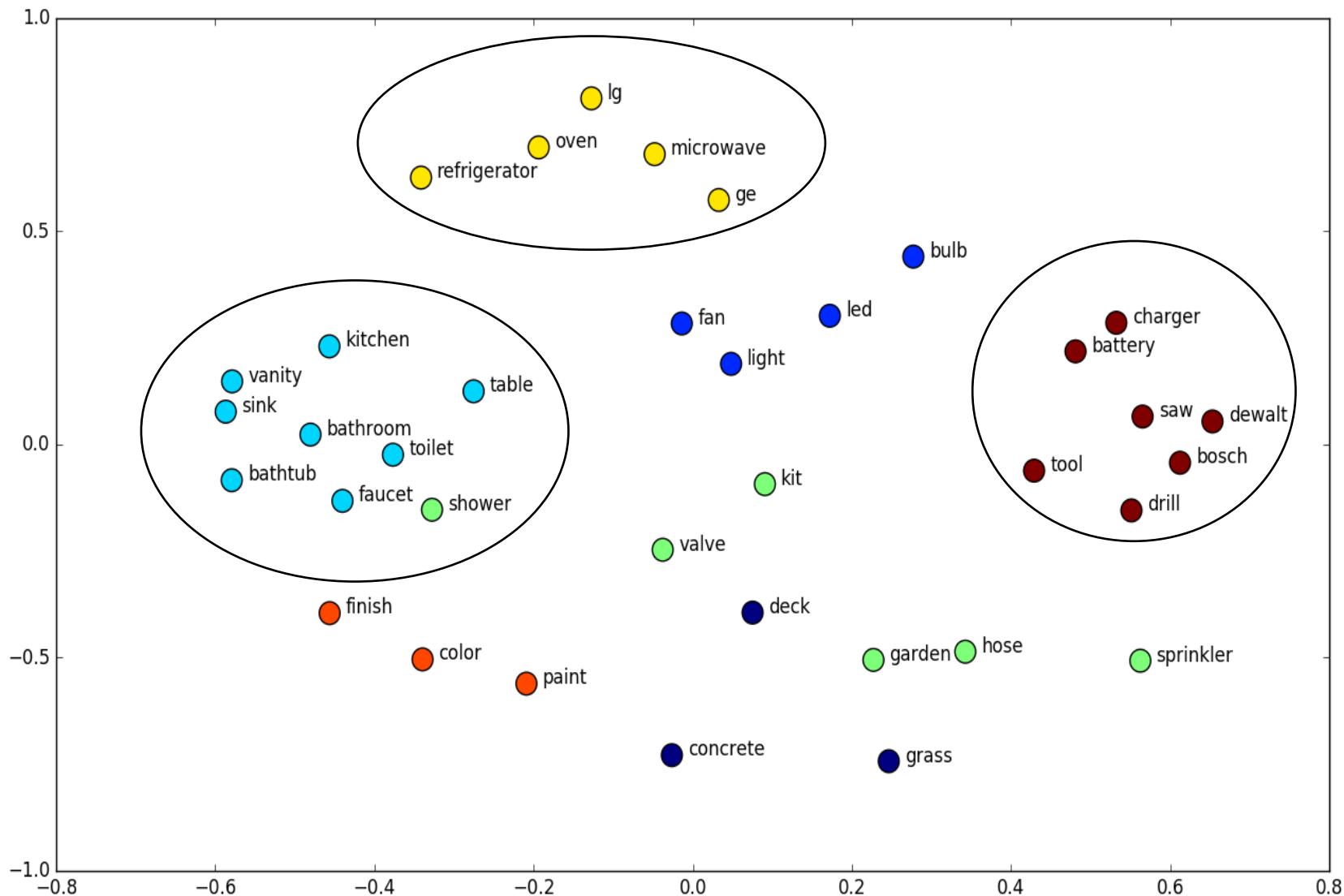
Bag of Words issues

- Vocabulary dimension: stop words (pronouns, articles ecc.), genre and number differences. Every word is a different column.
- Huge data structure filled with a lot of 0s: some words appear in only a few documents
- Bag of words doesn't have information about synonyms, contraries, omonyms and semantics
- For classification tasks, some words can be more useful than others and, in this representation they all have the same relevance

Word embedding

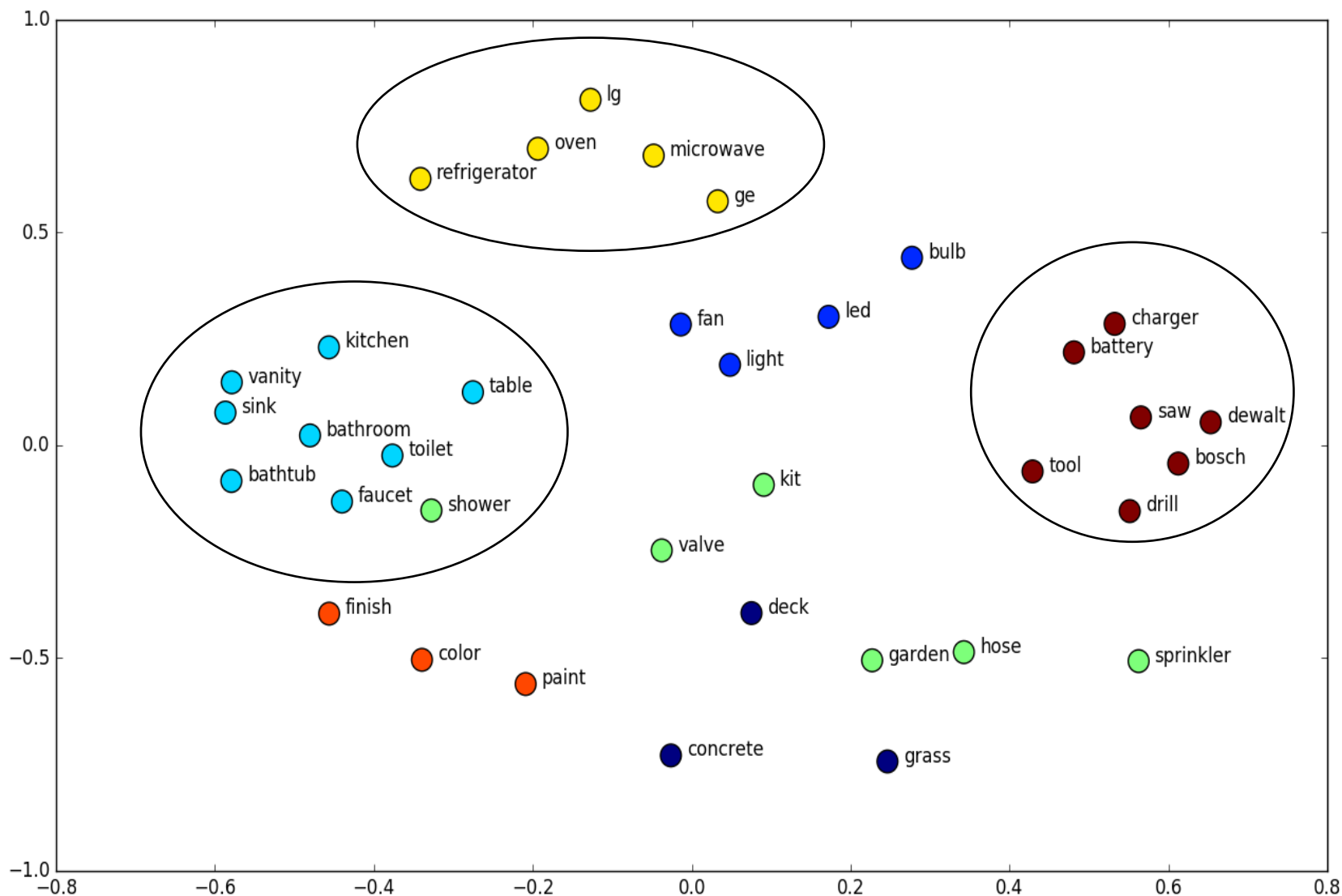
- Main idea: words are mapped into vectors of real numbers (also called **word vectors**)
- Every word is a point in a vector space
- We can calculate the distance between two points:
 - **Similar words** have **low distance** (synonyms, same topic)
 - **Unrelated words** have **high distance**
- Different ways and algorithms but neural networks are always involved

Word embedding (2D visualization)



- Words are a point in a two-dimensional space obtained with dimension reduction techniques
- Similar words are close (for example *bathroom-toilet-bathtub-sink*)
- Unrelated words are distant (for examples *microwave* and *grass*)

Word embedding in 2024

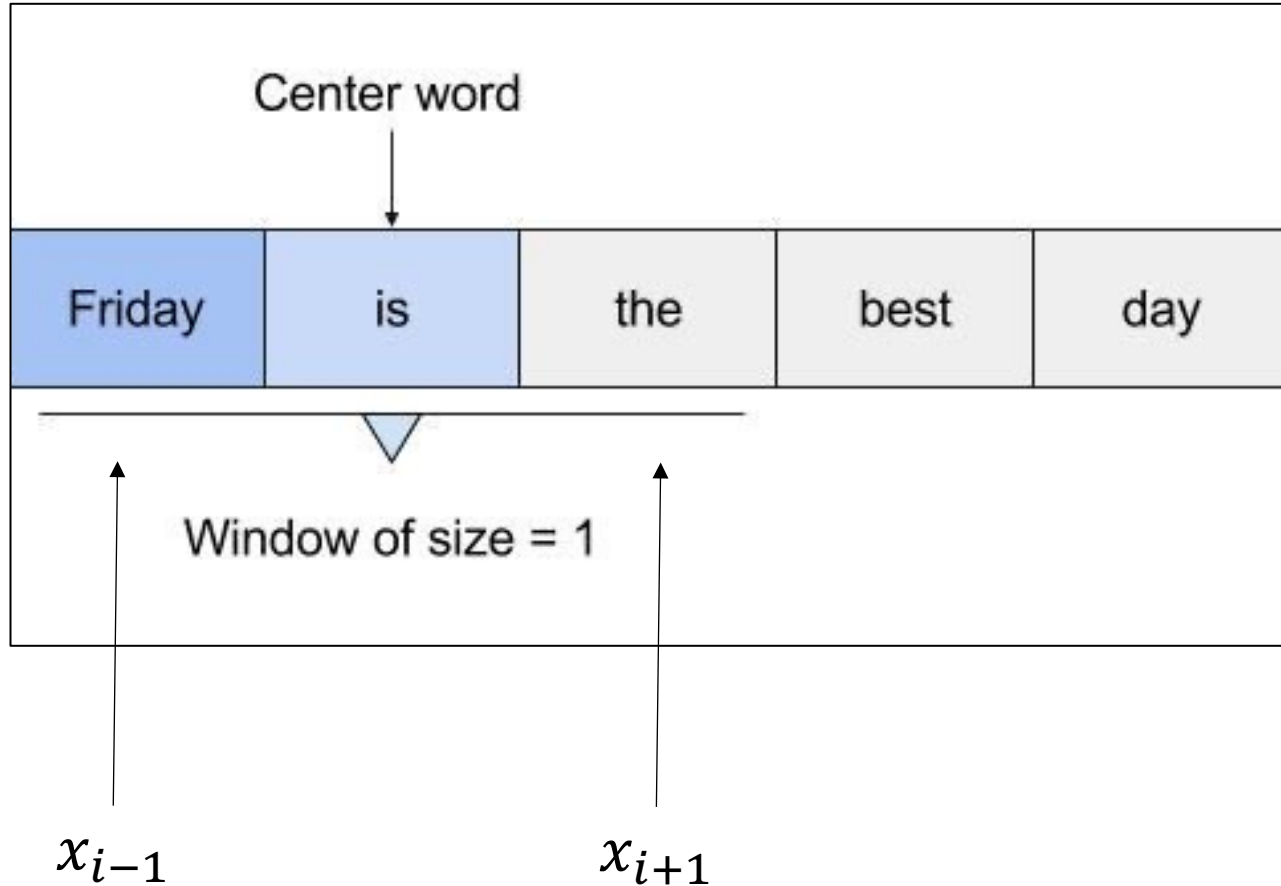


- This idea is not particularly new: it has been around since 2003
- But still, it's the basic foundation on everything that has been done in NLP, ChatGPT included.
- What has been changed is the type and the size of the neural network involved

Word embedding: implementation

- A neural network processes all the documents (ex. Wikipedia dump, news articles) trying to *understand how language works*. This procedure is also called **language modeling**.
- It takes a lot of time but, once the training is finished, we can use the word vectors learned by the network
- Main implementations: Word2Vec, Doc2Vec, FastText, GloVe

Word2Vec (Google, 2013)



- Idea: the words before and after a center word form the context
- Context is the neural network input
- Neural network learns how to predict the center word

Word2Vec - CBOW

- Also called «Continuous bag of words» (CBOW)
- Gives a vector space representation of all words that appear in the input documents
- **Vector size: from 100 to 1000**
- The window size is a parameter (often between 2 and 5)

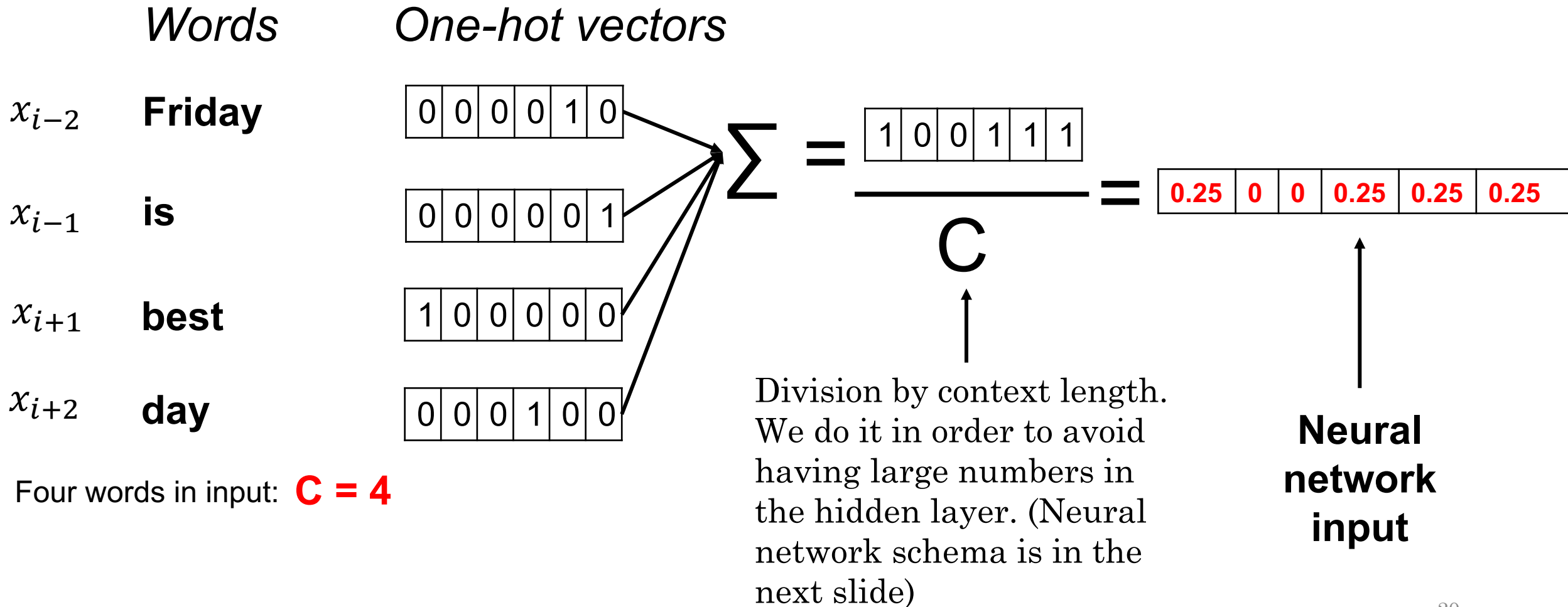
Word2Vec

Predicting **the** from *Friday is the best day* using **window-size = 2** and **vocabulary of 6 words**

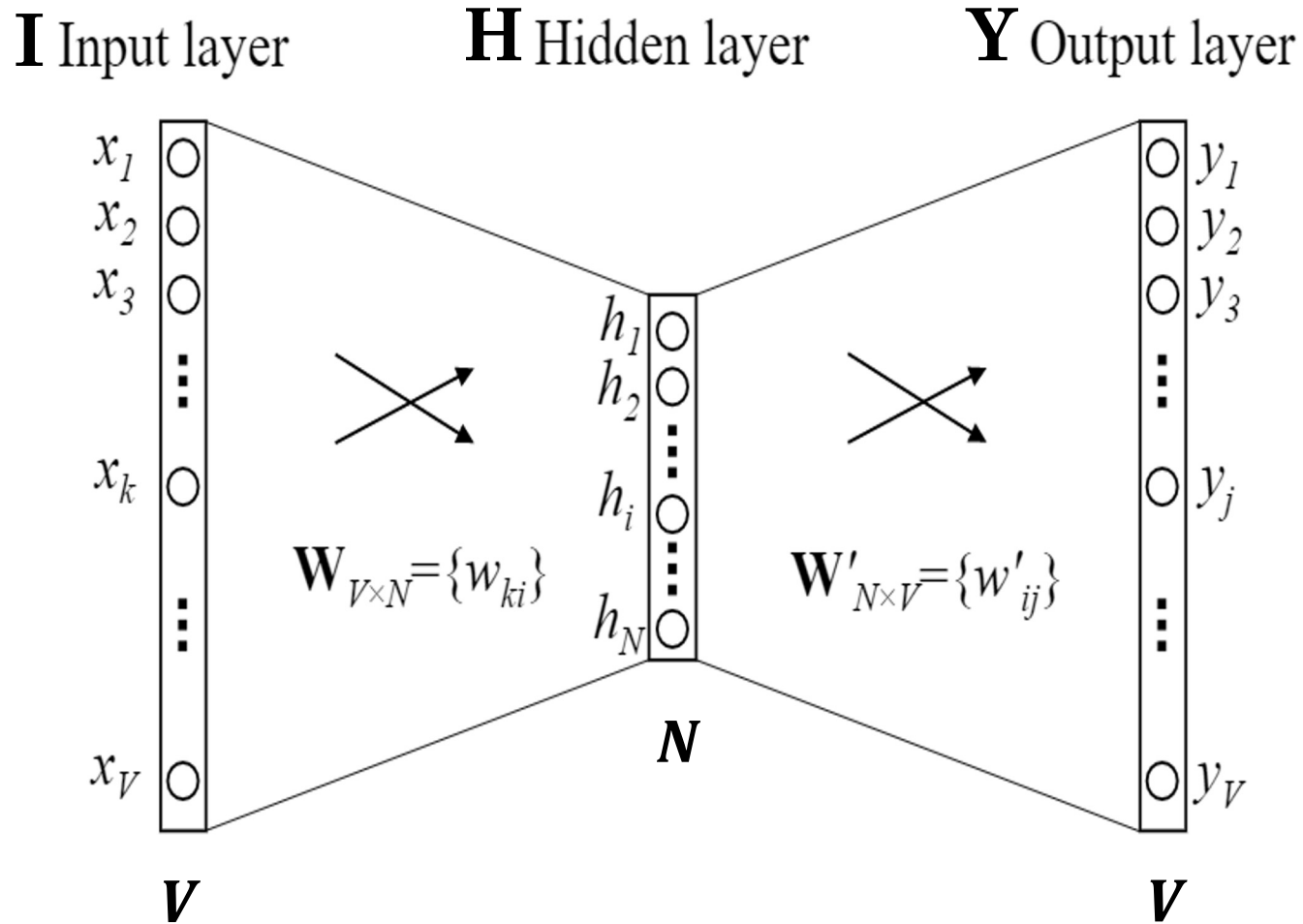
	Words	One-hot vectors							
x_{i-2}	Friday	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td></tr></table>	0	0	0	0	1	0	Words in input
0	0	0	0	1	0				
x_{i-1}	is	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	0	0	0	0	1	
0	0	0	0	0	1				
x_{i+1}	best	<table><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	0	0	0	0	0	
1	0	0	0	0	0				
x_{i+2}	day	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>	0	0	0	1	0	0	
0	0	0	1	0	0				
x_i	the	<table><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	1	0	0	0	Word in output
0	0	1	0	0	0				

Word2Vec - Input

Predicting **the** from *Friday is the best day* using **window-size = 2**



Word2Vec - Structure



V = Vocabulary dimension

N = Vector size

$$\mathbf{H} = \mathbf{I} \mathbf{W}$$

$$(V \times 1) * (V \times N) = N$$

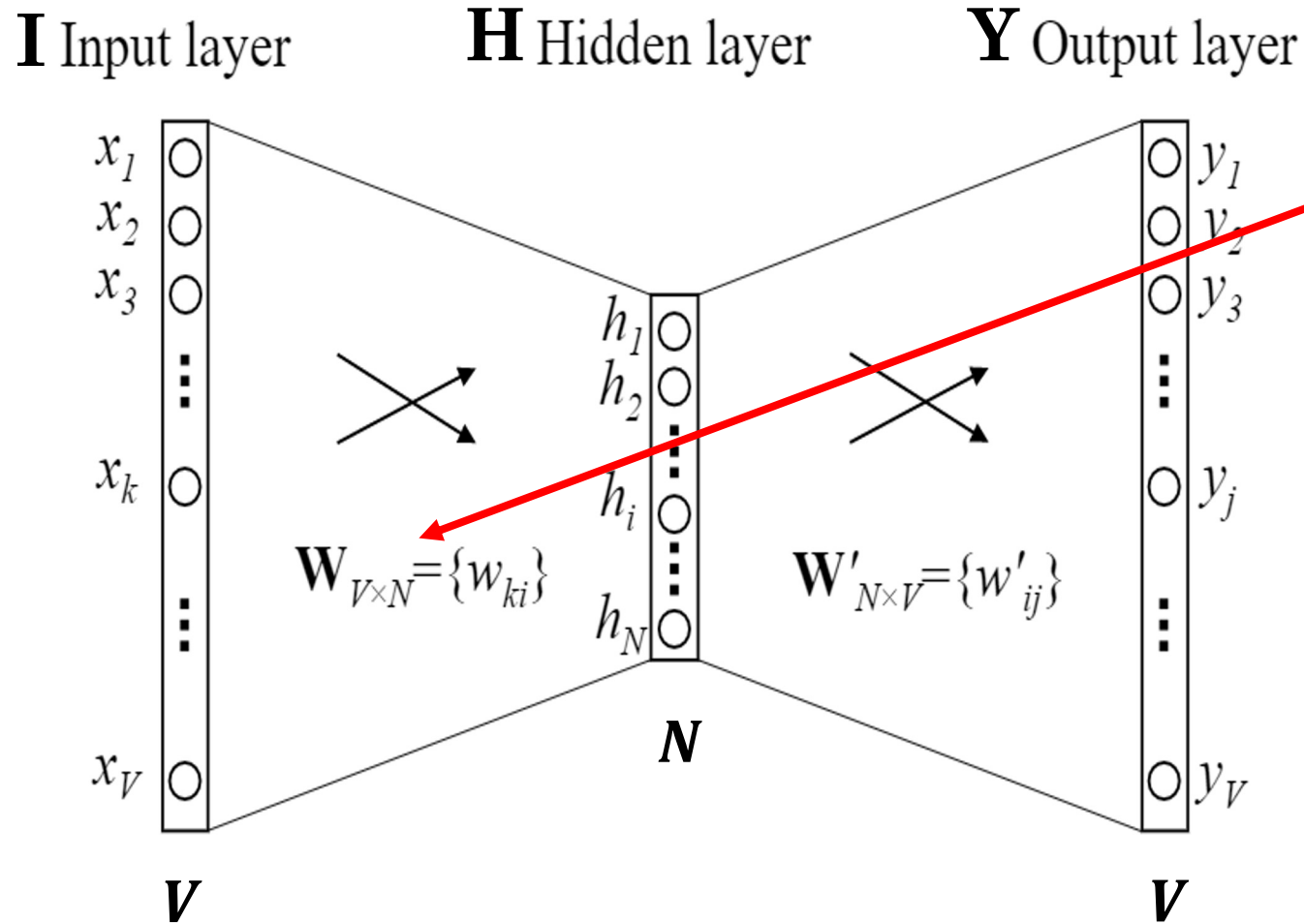
Hidden layer has a linear activation function $f(\mathbf{x}) = \mathbf{x}$

$$\mathbf{U} = \mathbf{W}' \mathbf{H}$$

$$(N \times 1) * (N \times V) = V$$

$$\mathbf{Y} = \text{Softmax}(\mathbf{U})$$

Where are the word vectors?



The weights between I and H are the word vectors

x_i is fully connected with the hidden layer: N weights

x_i can be represented as a vector of N real numbers using these weights

Softmax activation function (σ)

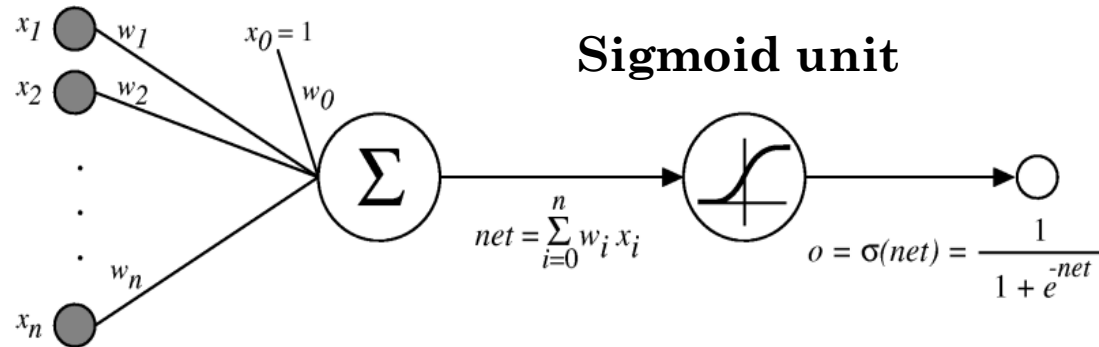
Definition: *takes an input of K real numbers and normalizes it into a probability distribution of K probabilities.* In our case, $K = [V]$ (vocabulary dimension)

$$\sigma(u)_j = \frac{e^{u_j}}{\sum_{i=1}^K e^{u_i}} \quad \sum_{j=1}^K \sigma(u)_j = 1$$
$$0 \leq \sigma(u)_j \leq 1 \quad \forall j$$

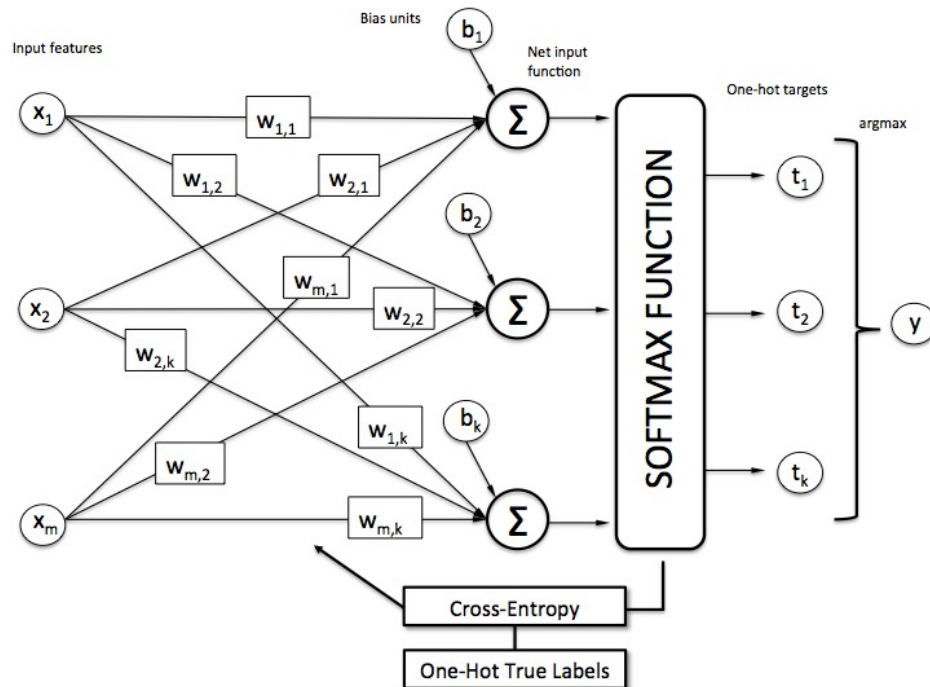
It's very used in multi-class classification because increasing one value in the output layer automatically decreases all the others (sum must be equal to 1).

Our case is a multi-class classification with V (vocabulary dimension) possible classes!

A new loss function



Backpropagation
algorithm as seen
in the neural
network lectures



See next slide.

Word2Vec Loss Function

1. Our goal is to predict w_p . The index of w_p is j^* and we want to have an output made by all zeroes except for a 1 in j^* . Therefore, we have to maximise y_{j^*} .

2. Maximising y_{j^*} is the same of maximising $\log y_{j^*}$ (technical trick...)

3.
$$\log y_{j^*} = \log \frac{e^{u_{j^*}}}{\sum_{i=1}^K e^{u_i}} = u_{j^*} - \log \sum_{i=1}^K e^{u_i}$$

4. In order to have a loss function to minimise, we consider:

$$E = -e^{u_{j^*}} + \log \sum_{i=1}^K e^{u_i}$$

5. E is convex so, calculating its gradient, we can apply gradient descent and use backpropagation algorithm: [here](#)'s all the math.

Word2Vec – Our toy example

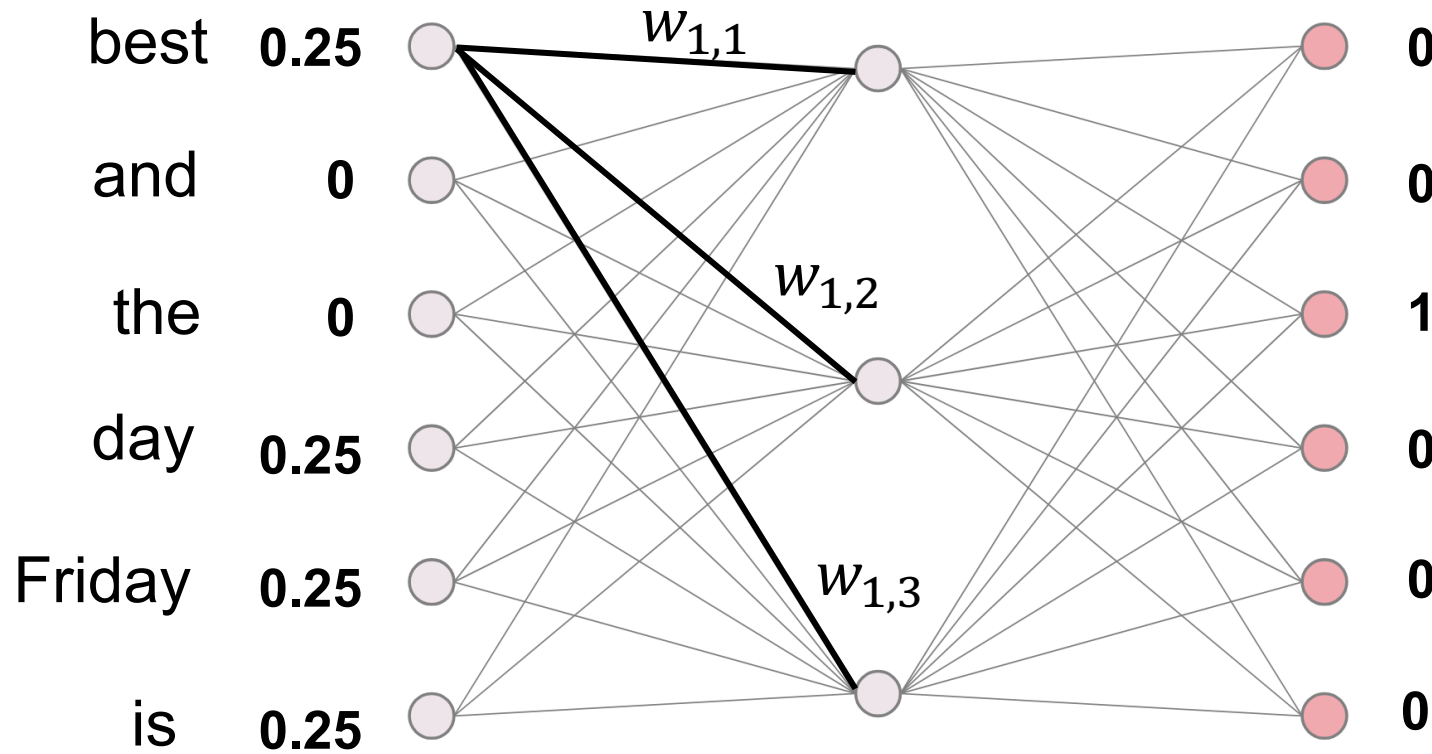
Predicting **the** from *Friday is the best day* using **window-size = 2** and **vector-size = 3**

We have already calculated the input vector:

0.25	0	0	0.25	0.25	0.25
------	---	---	------	------	------

And we have the goal (one-hot vector of **the**):

0	0	1	0	0	0
---	---	---	---	---	---



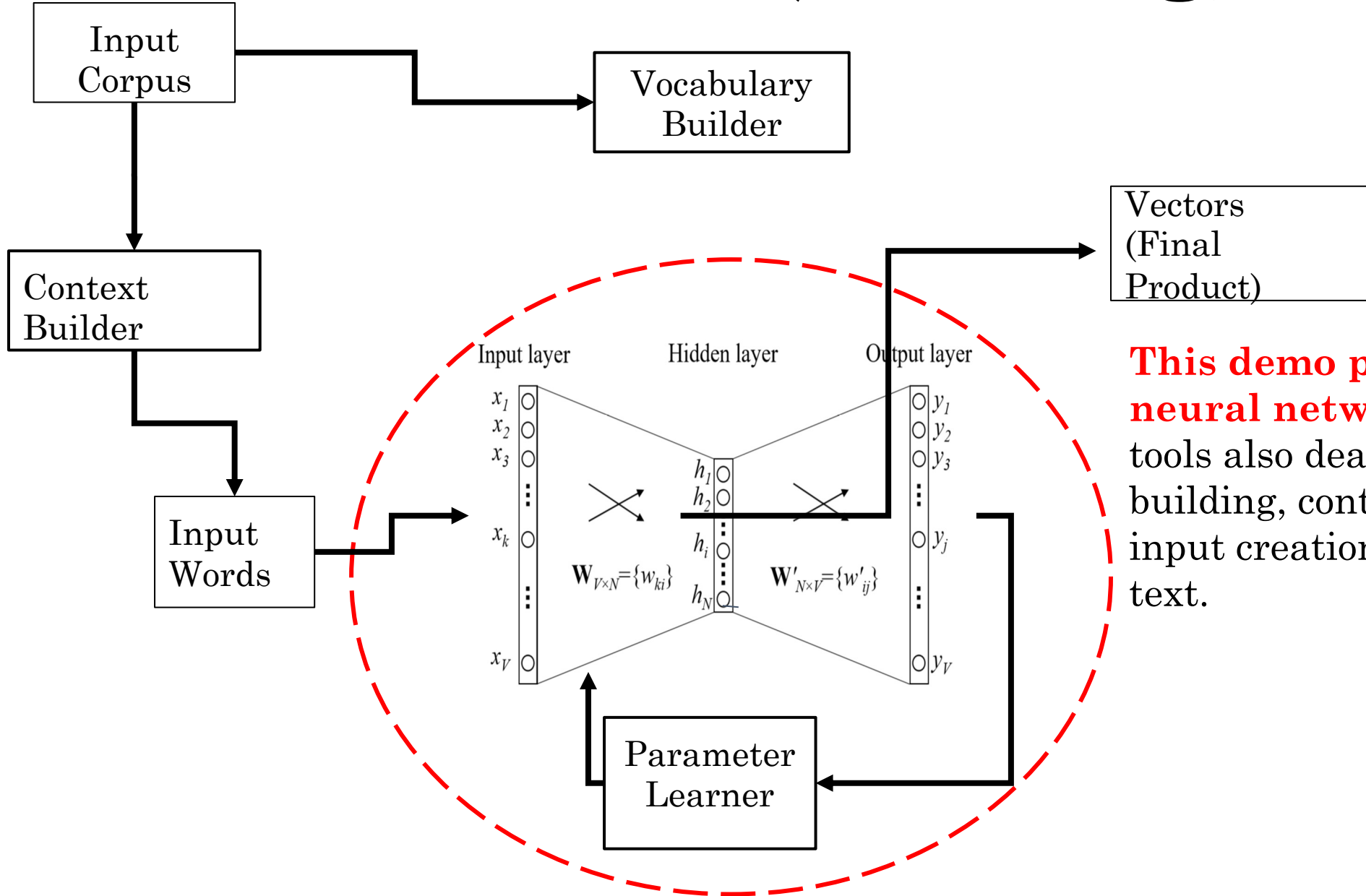
«best» one-hot vector:

1	0	0	0	0	0
---	---	---	---	---	---

«best» word vector:

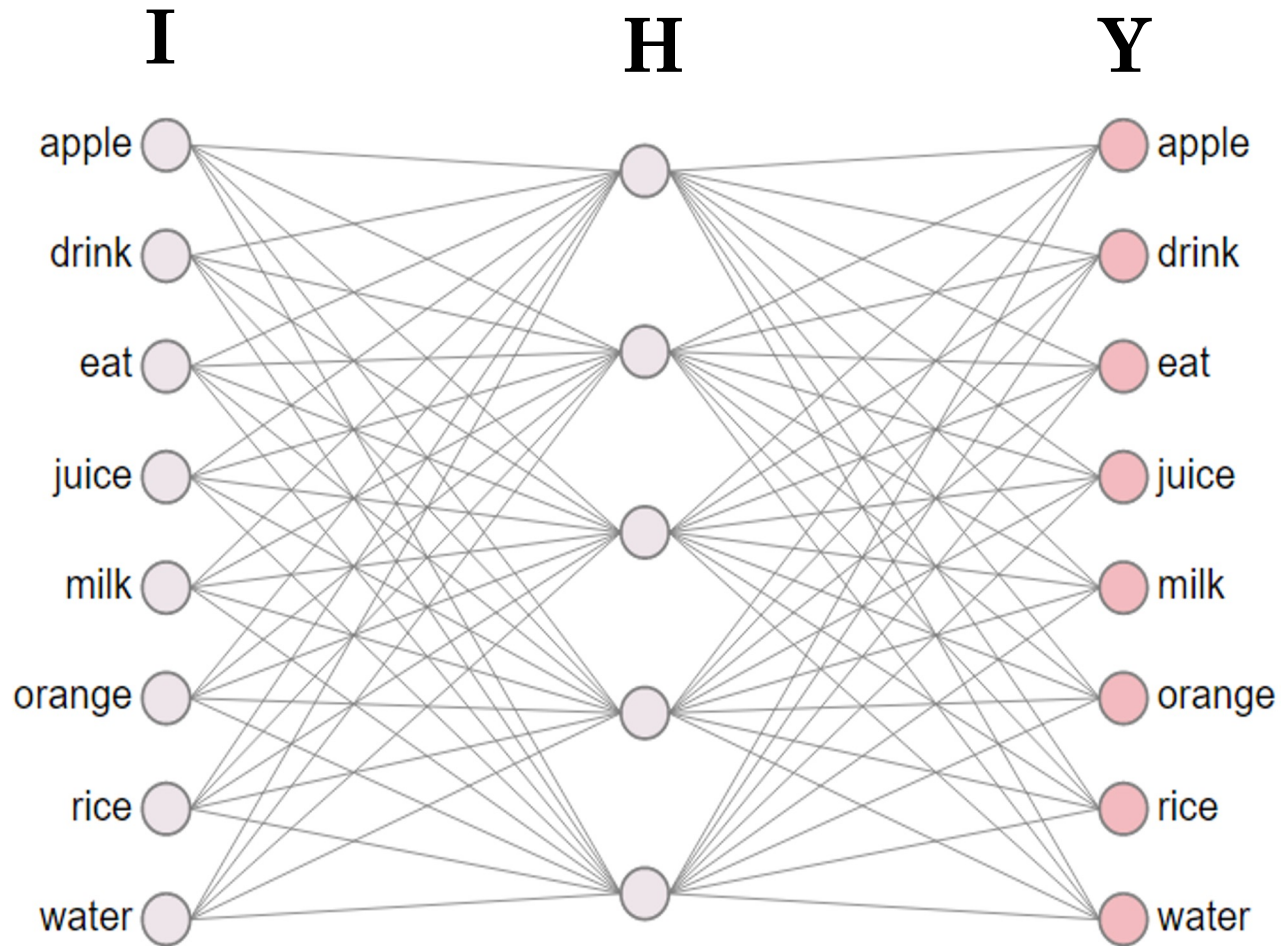
$[w_{1,1}, w_{1,2}, w_{1,3}]$

Word2Vec Demo (Xin Rong, 2016)



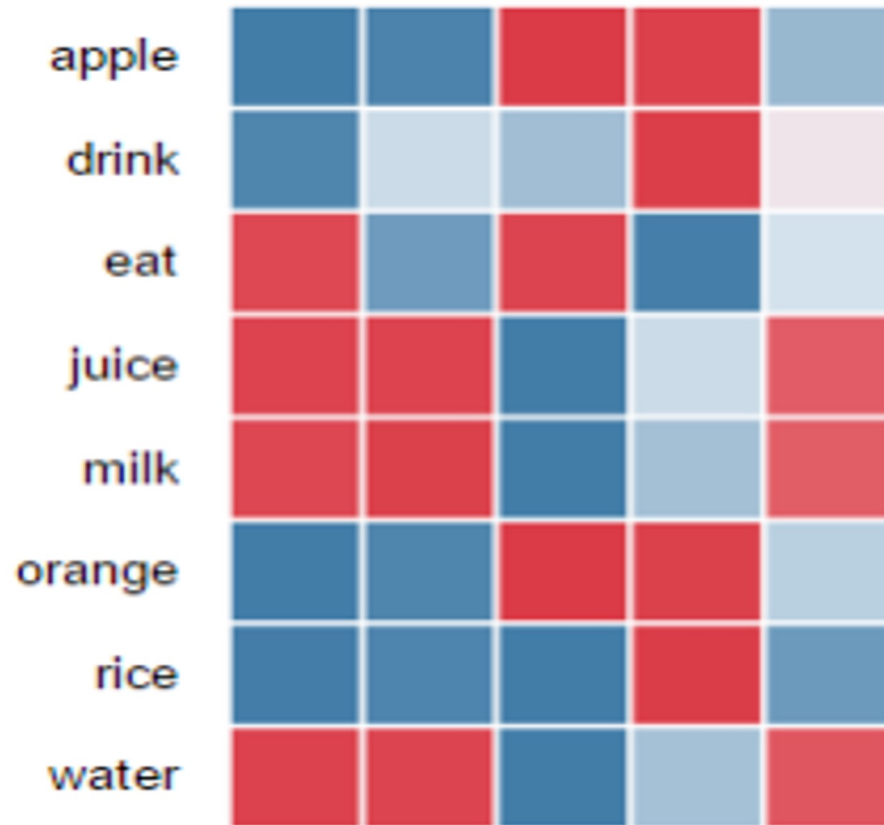
This demo presents only the neural network part. Word2Vec tools also deal with vocabulary building, context selection and input creation, starting from raw text.

Word2Vec Demo (Xin Rong, 2016)



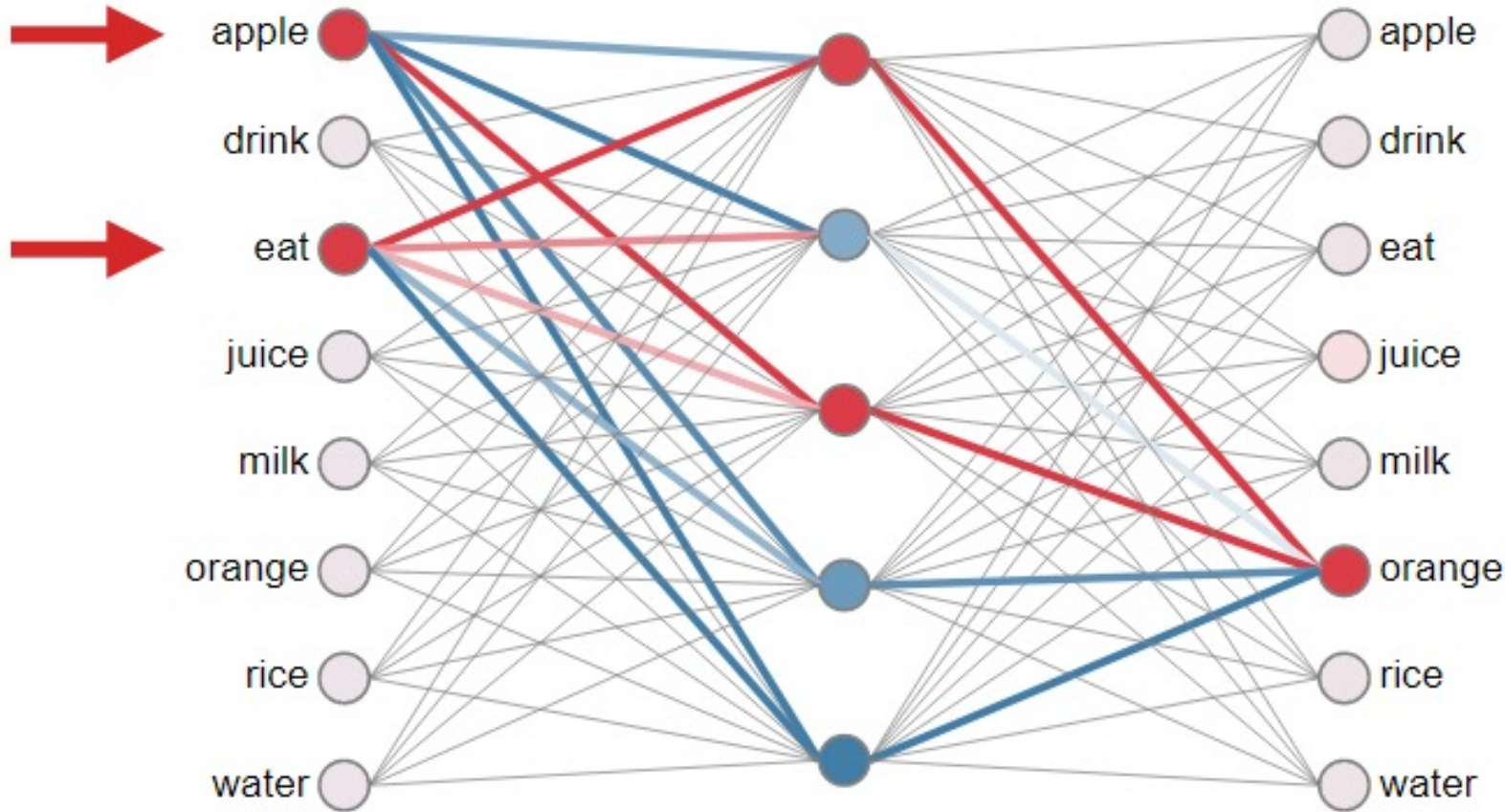
- <https://ronxin.github.io/wevi/>
- Input layer
 - One-hot vector
- Hidden layer: identity function per activation
 - $f(x) = x$
- Output layer
 - Softmax

Embedding Representation



- User can select dimension, training data, learning rate etc.
- This demo uses colors to visualize word vector values
- In this example, 8 words are represented in a vector with length = 5
- In reality, thousands of words are represented in a vector with length = 100

Word2Vec Demo



- The edges between «apple» and the hidden layer are the word vector representation of «apple»
- $C = \text{One word before} + \text{one word after} = 2$
- **apple^{eat} | orange:** uses *apple* and *eat* to predict *orange*

Pros

- Word2Vec is an efficient way to store natural language information: no more huge sparse matrixes but a fixed length
- Word2Vec performs better than Bag of Words in classification tasks
- There are free pre-trained word vectors available to download
- Language independent

Cons

- Results strongly depend on the number of documents that are used for training the word vectors
- A good training takes a lot of time
- Parameters: window-size? Neural network parameters? Is it the same of every language?

**How can we do text classification
using Word2Vec?**

Radiology report of a lung TC

TC TORACE CON E SENZA MDC

Esame eseguito con somministrazione di 90 ml di Iomeron 350, confrontato con precedente del 7/6/2017. Al controllo attuale nel segmento posteriore del lobo superiore destro è riconoscibile lesione nodulare di 15 x 15 mm con margini un poco irregolari e spiculati, localizzata in sede peri-ilare, a stretto contatto con sottili diramazioni vascolari e una diramazione bronchiale subsegmentaria che presenta pareti ispessite; a giudizio clinico utile broncoscopia. Minimo incremento del versamento pericardico. Immodificati i restanti rilievi.

Exam Type: Follow-up

Result: Stabile

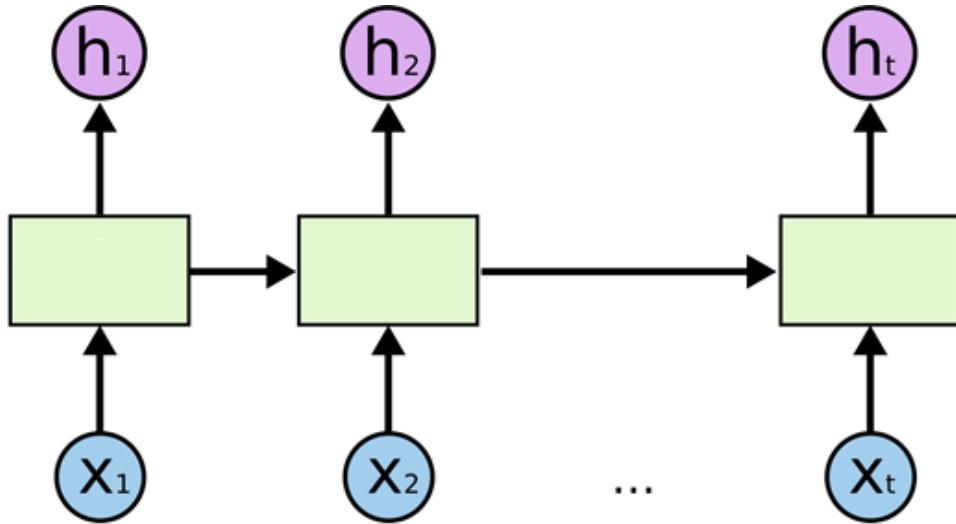
Lesion Nature: Neoplastica

Lesion Site: Polmone

Radiology reports classification

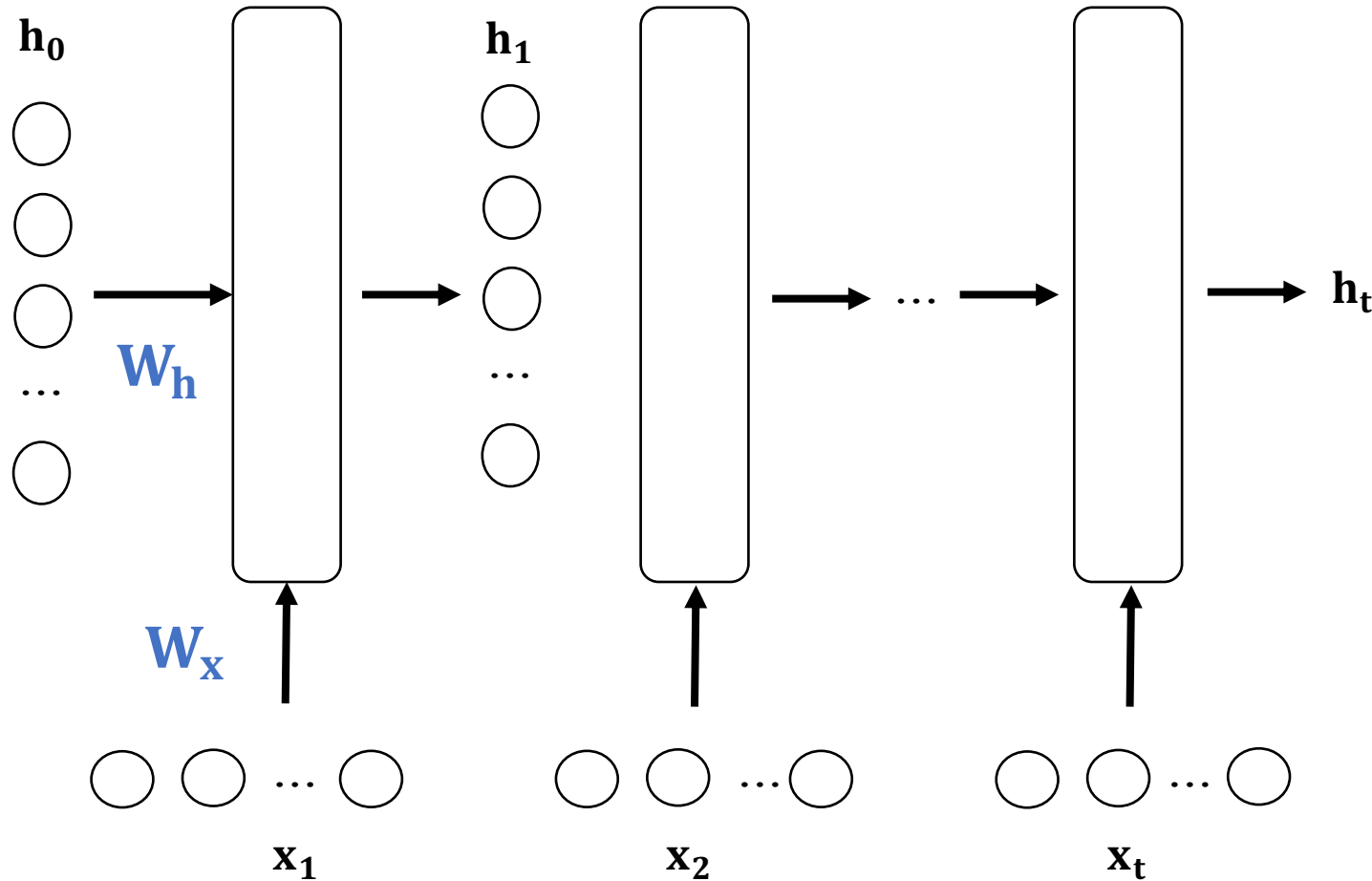
- Dataset: 5k classified reports, 10k unclassified obtained from Spedali Civili di Brescia
- We trained a Word2Vec model with all our reports with:
 - Window-size = 2
 - Vector size = 200
 - Epochs = 60
- If a report is composed by N words, we create a $N \times 200$ matrix to represent the entire document
- For each level, Recurrent Neural Network processes the document and provides the classification

Recurrent Neural Network



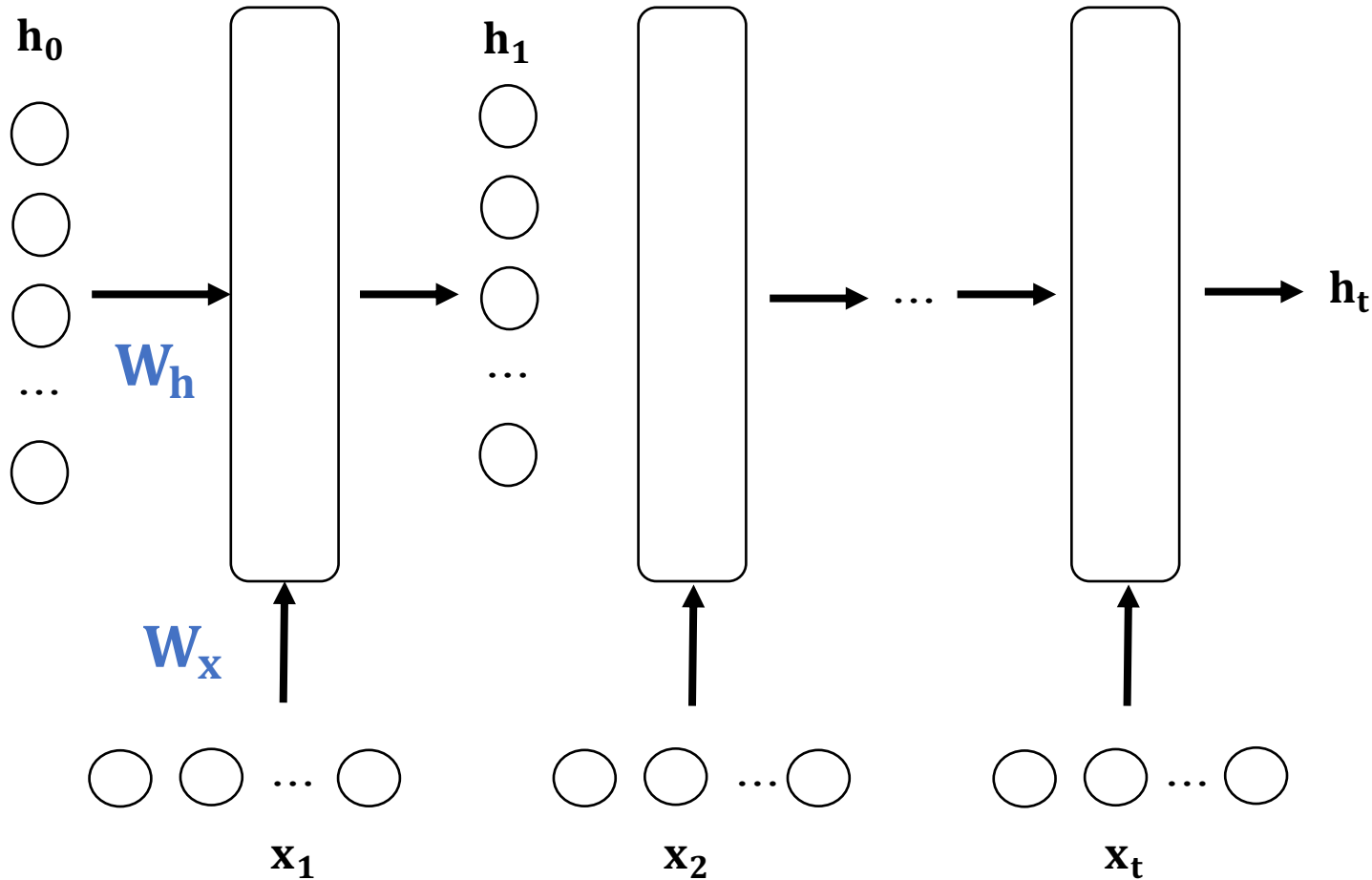
- Used for analyzing sequences (text, video, signals...)
- The output not only depends on the current input but also on the history and the previous input
- x_1, x_2 etc. are the embedding vectors representing the words (i.e. vectors of real numbers between -1 and 1)
- Given a document of t words, the RNN is made by t **cells**. Each word enters in its cell respecting the order of the document.
- Each cell produces an output **h**, which is another vector of real numbers.

Recurrent Neural Network



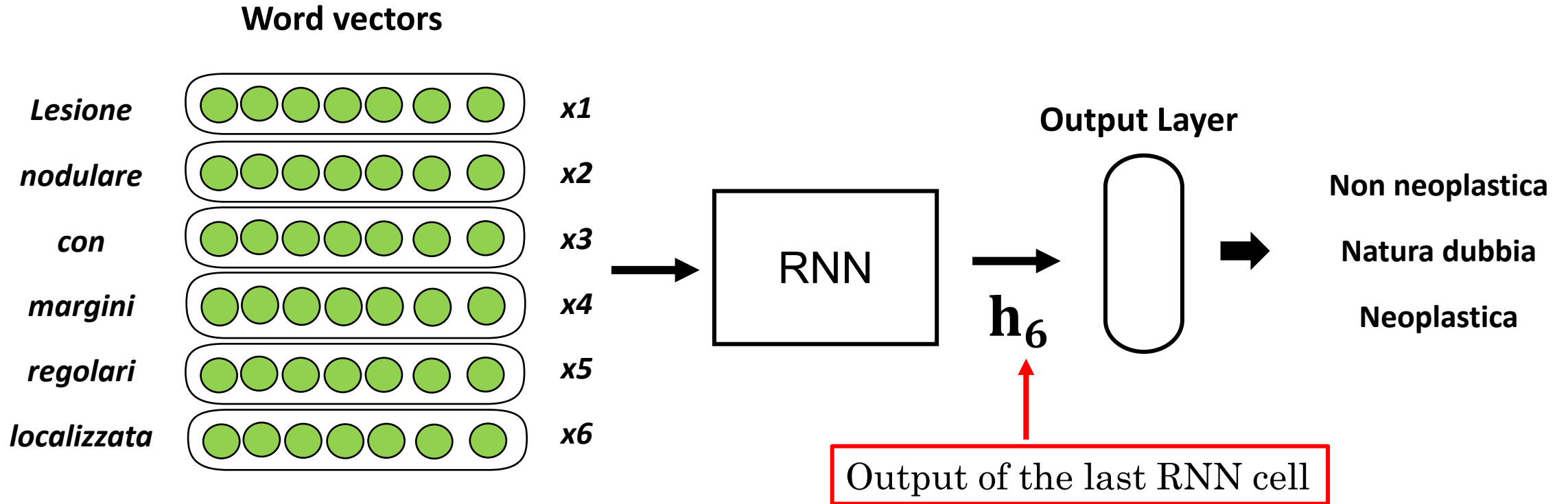
- Each cell is a fully-connected layer and takes x (the word vector) and h (the result of the previous cells) in input.
- Each h passes to the next layer and it is processed along with the next word
- W_h and W_x are the weight matrices of the fully-connected layers and they are shared by all the cells

Recurrent Neural Network



- The number of input neurons for x is fixed: in fact, it depends on the dimension of the word vector
- Instead, the number of neurons for h is an hyperparameter (like the number of neurons for a fully-connected neural network)
- h_0 is initialised as a vector of zeroes
- h_t represents the entire document and it can be used for classifying it

RNN-based model



In this case, the output layer is simply a fully-connected layer with 3 neurons: one for each class (Non neoplastica, Natura Dubbia, Neoplastica)

Results

ACCURACY

	Annotazioni + ML	Word2Vec + RNN
Tipo Esame	96,0	96,2
Risultato Primo Esame	77,3	78,3
Risultato Follow-Up	73,9	81,9
Tot. Risultato Esame	74,7	80,1
Natura Lesione	66,3	73,2

DETTAGLIO TERZO LIVELLO (F-SCORE)

	Annotazioni + ML	Word2Vec + RNN
Non Neoplastica	72,6	73,5
Natura Dubbia	43,1	57,2
Neoplastica	66,9	81
Media	60,9	71,2

Conclusions

- Natural Language Processing tasks and some of its challenges
 - Classification, Sentiment Analysis, Word Representation
 - Genre, number, synonyms, contraries...
- Bag of Words
- How word vectors work and Word2Vec algorithm
- Introduction to RNNs and their application for the classification of radiology reports

What's next?

- From 2018-2019, new architectures became predominant
- These architectures (BERT, GPT, T5, ChatGPT) are based on a complex deep learning model called Transformer
- However, most of the ideas presented here are still valid and exploited by these architectures
- More information about these new models in the Deep Learning and **Architectures of Neural Network** courses. For who is interested, see you next semester.

Thank you